

QuantZone 因子分组回测

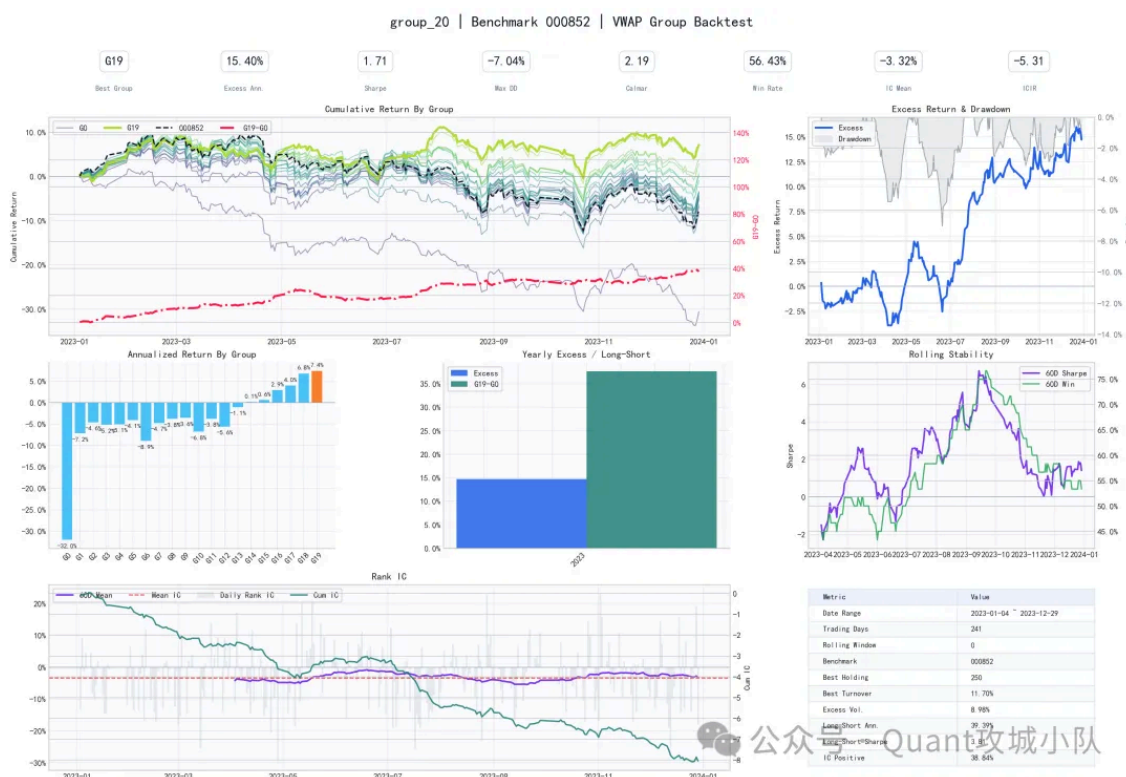
天陽小七 Quant攻城小队 2026年7月12日 12:45 上海

过去一段时间，小编更新频率确实少了很多，先跟一直关注我们的读者朋友说声抱歉。不是偷懒，也不是忘了这个号，而是把大部分时间都投入到了 QuantZone 量化 SDK 的筹备和打磨里。我们希望它不只是几篇文章里的代码示例，而是能真正帮大家少踩坑、少重复造轮子：更方便地接入数据、查询因子、整理投研流程，把时间更多留给策略研究和交易想法本身。

这篇文章先从大家最常用、也最容易写复杂的环节讲起：因子分组回测。本文不只讲参数，也用 dtws_20 因子的真实回测产物，把分组收益、Rank IC、超额指标、成本敏感性、股票池和并行耗时都跑一遍。本文示例仅用于展示 SDK 工作流，不构成任何投资建议。

本次实测设置

- SDK 版本：quantzone 0.9.0，PyPI 当前最新版本：0.9.0。
- 因子：dtws_20。
- 因子矩阵：242 个交易日 × 5121 只股票。
- 回测区间：2023-01-01 至 2023-12-31。
- 默认基准：000852，默认分组数：20。



这张图是最直观的第一眼检查：如果因子有稳定区分度，通常希望不同分组之间能拉开差距，而不是所有线条缠在一起。本次样例中，SDK 判定的最佳组是 19。

为什么要把分组回测做成一个 SDK 函数

手写因子分组回测并不难，难的是长期保持同一套严谨口径。一个完整流程至少要处理日期和股票代码对齐、分组数量、排序方向、股票池、成交约束、交易成本、基准超额、Rank IC、最大回撤、胜率和图片输出。

`qz.backtest` 的设计目标，就是把这些重复、容易遗漏、又必须统一的环节封装起来。用户只需要准备一个宽表因子矩阵，再指定回测区间和参数，就可以得到标准化结果对象。

最小可运行回测

核心流程只有六步：初始化 SDK、加载本地回测基础数据、查询因子、转宽表、调用 `qz.backtest`、读取 `result.summary()`。

```
import quantzone as qz

qz.init(
    access_key="your_access_key",
    sign_secret="your_sign_secret",
    base_url="https://api.quantzone.tech",
)

backtest_data = qz.load_backtest_data()
df = qz.get_factors(factor="dtws_20", start_date="2023-01-01", end_date="2023-12-31")

factor = df.pivot(index="date", columns="order_book_id", values="value")
factor.index = factor.index.astype(str).str.replace("-", "", regex=False)
factor.columns = factor.columns.astype(str).str.slice(0, 6)

result = qz.backtest(
    factor,
    backtest_data,
    "2023-01-01",
    "2023-12-31",
    "./output/dtws_20",
    "dtws_20",
    group_num=20,
    benchmark="000852",
    plot="large",
)

print(result.summary())
```

```
print(result.best_group)

print(result.plot_paths)
```

默认参数下，000852 基准对应的结果摘要如下：

基准	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
000852	15.40%	1.71	-7.04%	2.19	56.43%



输入因子 factor 应该长什么样

factor 是一个宽表 DataFrame：index 是交易日期，建议统一成 YYYYMMDD 字符串；columns 是 6 位股票代码；values 是对应日期、对应股票的因子值。

	000001	000002	600519
20230103	0.1254	-0.0312	0.8841
20230104	0.1198	-0.0288	0.9020
20230105	0.1410	-0.0127	0.8765

这里要强调一个经验：回测前先把日期和股票代码标准化。很多“结果为空”“股票对不上”的问题，本质上不是回测逻辑错，而是输入矩阵的 index 或 columns 没有对齐。

参数逐个讲

qz.backtest 当前签名如下：

```
qz.backtest(
    factor,
```

```

data,
start_date,
end_date,
output_dir,
plot_filename,
*,
group_num=20,
benchmark="000852",
universe=None,
rolling_window=0,
ascending=False,
total_money=1e8,
fee=0.0001,
rule_fee=0.0005,
plot="long",
)

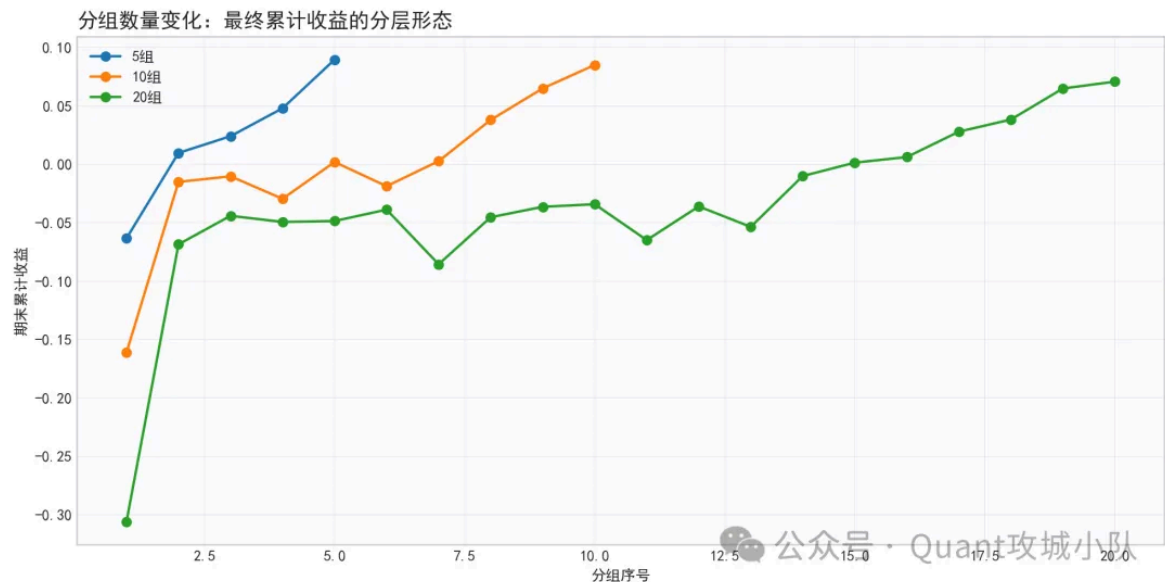
```

`plot_filename` 后面的参数都是关键字参数，建议显式写出参数名，避免以后调整顺序时误传。

调整分组数量 `group_num`

分组数量决定了因子分层的细腻程度。 `group_num=5` 更粗、更稳定，适合快速观察单调性； `group_num=10` 是常见十分组； `group_num=20` 更细，适合观察头尾组区分度。

场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
5组	000852	4	17.37%	2.36	-5.17%	3.36	58.09%
10组	000852	9	16.89%	2.02	-6.13%	2.76	57.26%
20组	000852	19	15.40%	1.71	-7.04%	2.19	56.43%



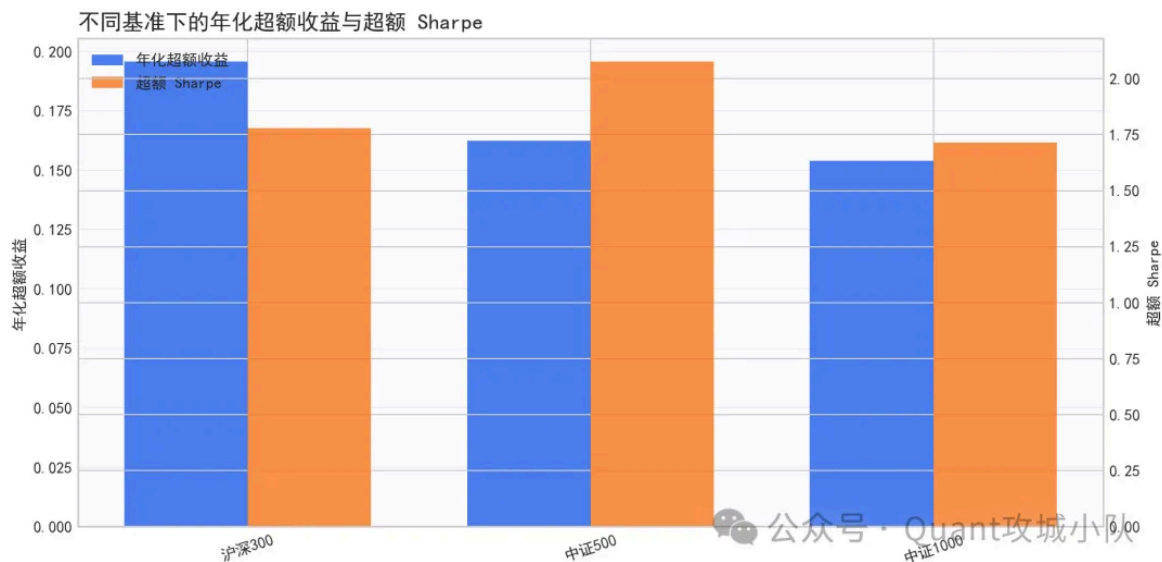
分组越多，头尾组差异可能更明显，但每组股票数量更少。分组太细时，要额外关注成交约束和样本稳定性。



切换基准 benchmark

同一个因子，在不同基准下的超额表现可能不同。这里分别对沪深 300、中证 500、中证 1000 做比较。

场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
沪深300	000300	19	19.56%	1.78	-5.85%	3.34	55.60%
中证500	000905	19	16.23%	2.08	-5.68%	2.86	56.02%
中证1000	000852	19	15.40%	1.71	-7.04%	2.19	56.43%

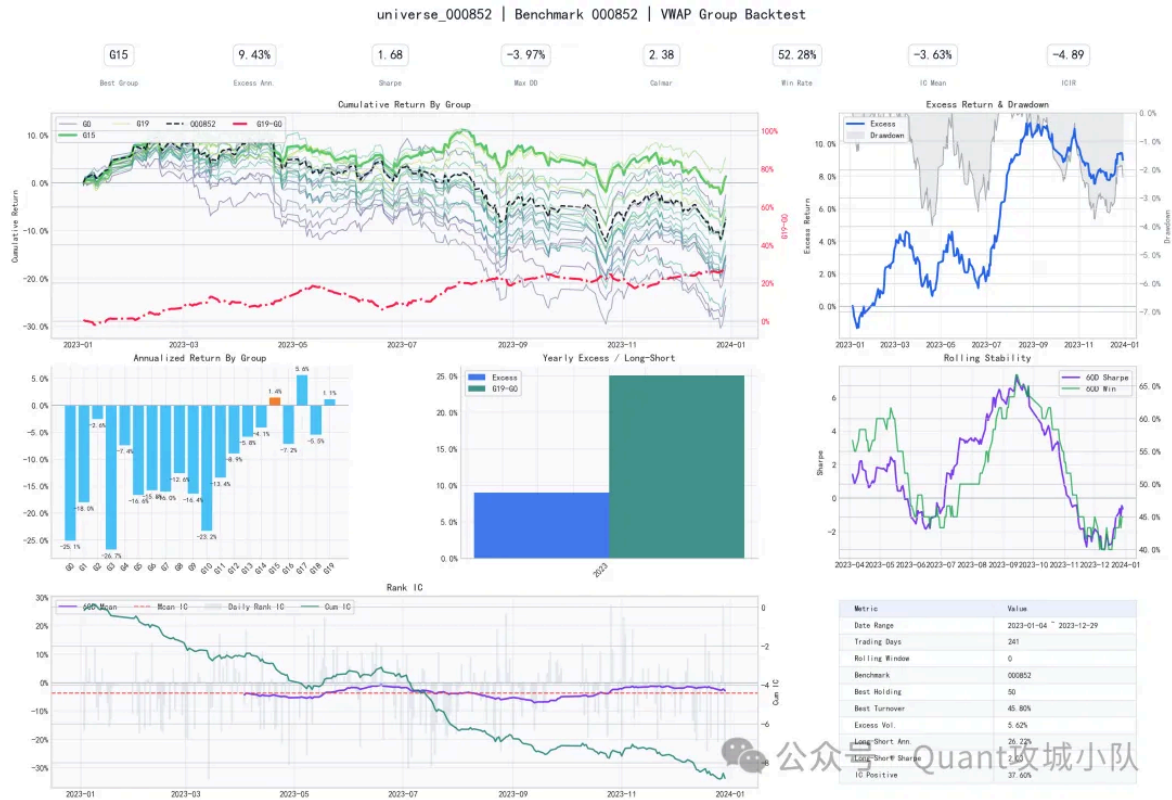


控制选股范围 universe

benchmark 是超额收益比较基准，universe 是参与选股和分组的股票范围。两者可以相同，也可以不同，取决于研究问题。

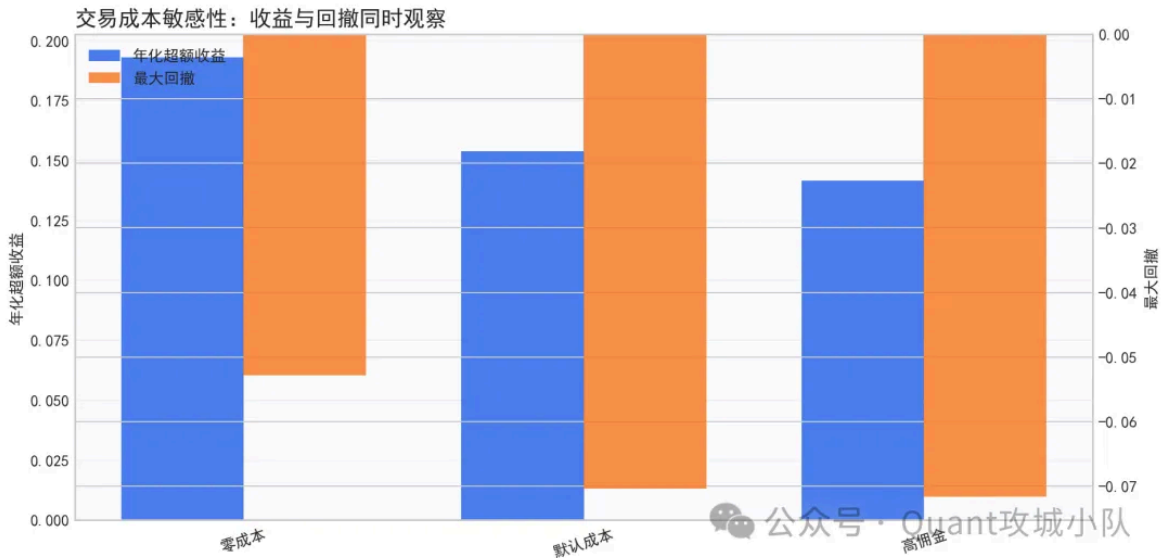
场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
全市场选股	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
中证1000内选股	000852	15	9.43%	1.68	-3.97%	2.38	52.28%

例如，在中证 1000 成分股内选股，并相对中证 1000 看超额，可以设为 universe="000852"、benchmark="000852"；如果在全市场内选股，但用中证 1000 做比较基准，则是 universe=None、benchmark="000852"。



交易成本敏感性测试

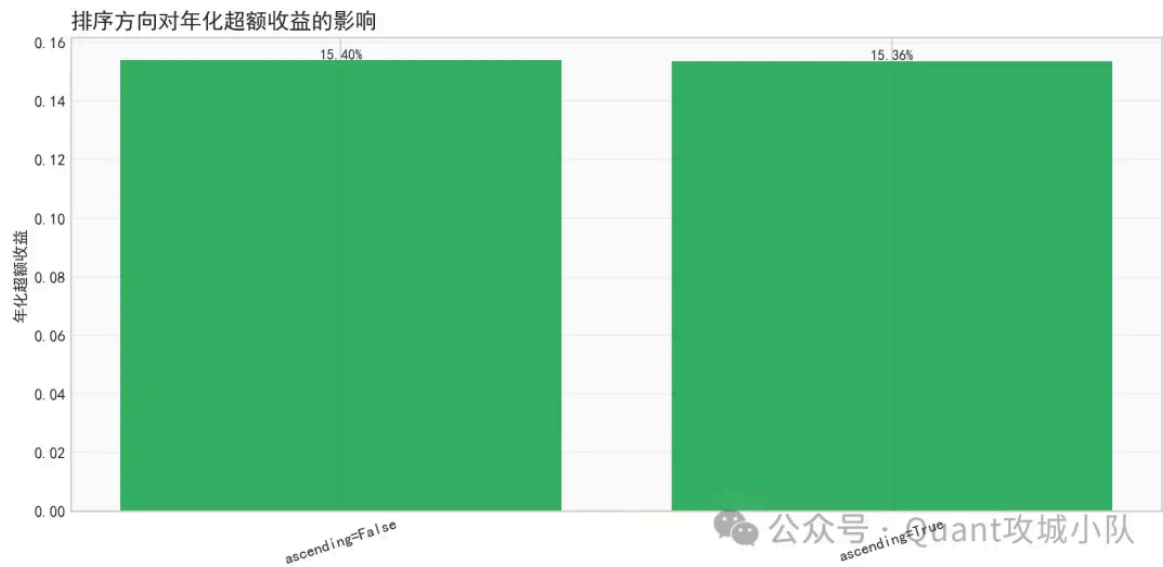
一个因子如果只在零成本下好看，实盘意义会打折。qz.backtest 把买卖佣金 fee 和卖出印花税 rule_fee 暴露出来，方便直接做敏感性测试。



场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
零成本	000852	18	19.31%	2.44	-5.28%	3.65	57.68%
默认成本	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
高佣金	000852	19	14.16%	1.58	-7.17%	1.98	56.02%

排序方向 ascending

不是所有因子都是“越大越好”。动量、资金流强度、趋势占优类因子可能是值越大越好；估值、波动、拥挤度、风险暴露类因子可能是值越小越好。



场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
ascending =False	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
ascending =True	000852	0	15.36%	1.71	-7.01%	2.19	56.43%

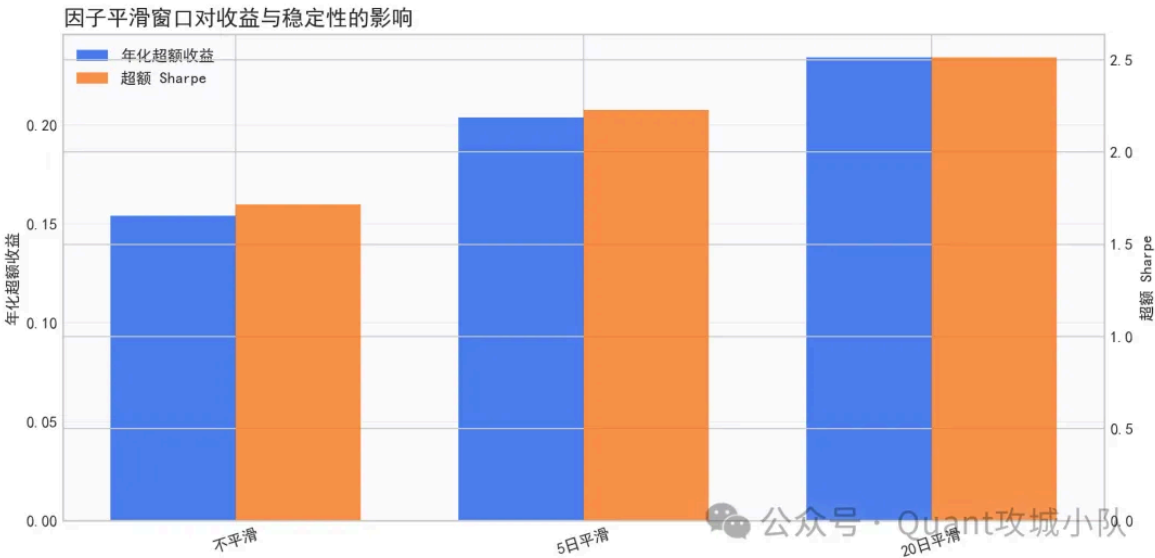
如果最佳组突然从尾部切到头部，通常意味着要重新审视因子方向、经济含义和样本期表现。



因子平滑 rolling_window

rolling_window 可以对因子做滚动均值处理，降低短期噪声，但也可能降低因子的及时性。

场景	基准	最佳组	年化超额收益	超额 Sharpe	最大回撤	Calmar	胜率
不平滑	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
5日平滑	000852	19	20.39%	2.23	-6.94%	2.94	57.81%
20日平滑	000852	19	23.41%	2.51	-7.82%	2.99	59.01%

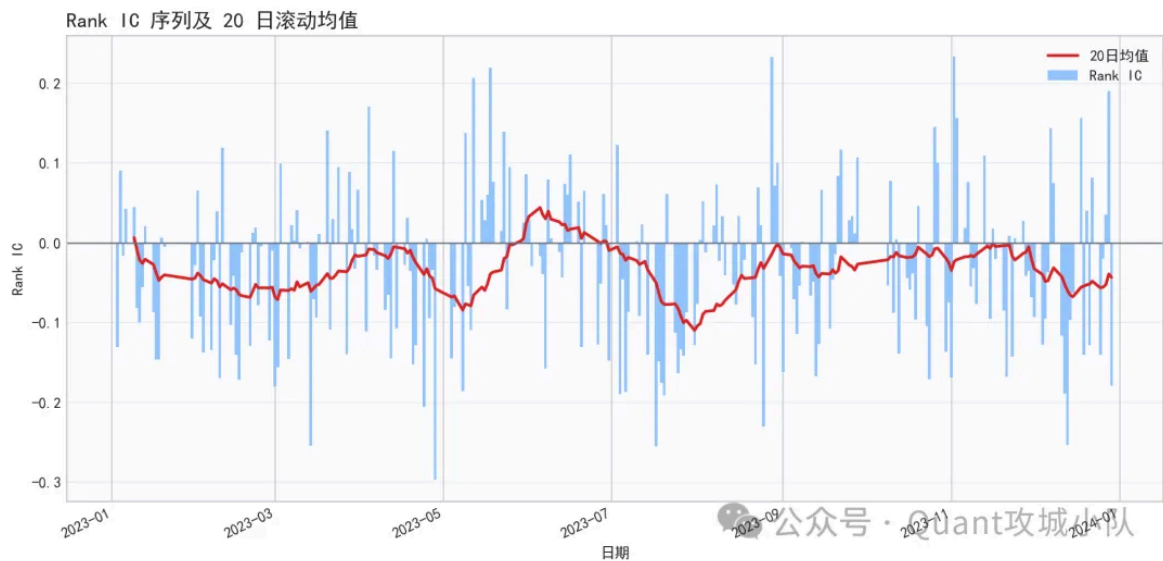


是否平滑，不应该只看表格里哪个数更高，还要结合因子本身的经济含义：它是日频快速信号，还是更慢的状态变量？

结果对象 BacktestResult 怎么看

qz.backtest 返回 BacktestResult，常用字段包括 best_group、group_daily_return、group_cumulative_return、rank_ic、benchmark_summary、plot_paths。这些字段都是标准 pandas 结构，适合继续保存、画图和批量比较。

Rank IC 的描述统计如下：



场景	count	mean	std	min	25%	50%	75%	max	positive_rate
rank_ic	242.0	-0.03	0.09	-0.29	-0.1	-0.03	0.03	0.23	0.388

公众号 · Quant攻城小队

算法优点一：回测口径完整

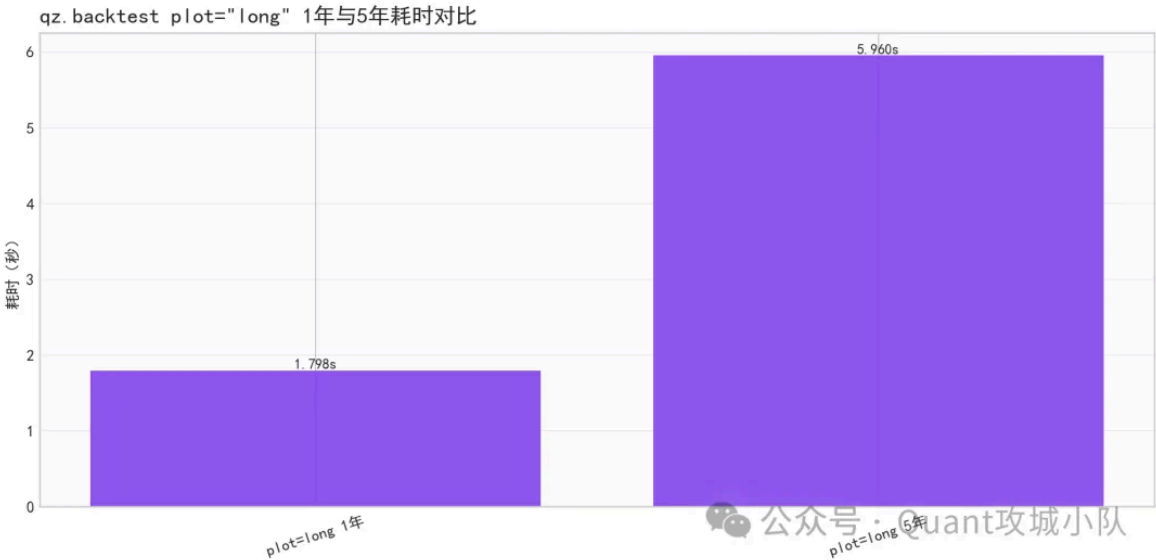
完整不是“指标很多”，而是把实盘前必须统一的假设都放到同一个入口里：交易费用、成交约束、股票池、基准、分组数量、排序方向、平滑窗口和图片输出。这样横向比较不同因子时，不会因为某个脚本漏了成本、另一个脚本换了股票池而误判。

算法优点二：计算速度快

为了避免“只算核心逻辑、不算出图”的偏差，本次脚本专门把 qz.backtest 的 PLOT_STYLE = "long" 情形单独跑了一遍。下面的耗时包含完整 dashboard 生成过程，因此更接近研究员日常真正等待的时间。

plot="long"; 1年 vs 5年耗时

case	factor	date_range	trading_days	stock_count	plot_style	elapsed_seconds	benchmark	best_group	annualized_excess_re	excess_sharpe	max_drawdown	calmar	win_rate
plot=long 1年	dtws_20	2023-01-01 2023-12-	242	5121	long	1.798s	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
plot=long 5年	dtws_20	2019-01-01 2023-12-	1213	5215	long	5.960s	000852	19	15.40%	1.09	-16.10%	0.81	56.89%



可以看到，5 年窗口会明显增加计算量，但耗时并不是简单“体感不可接受”式增长。

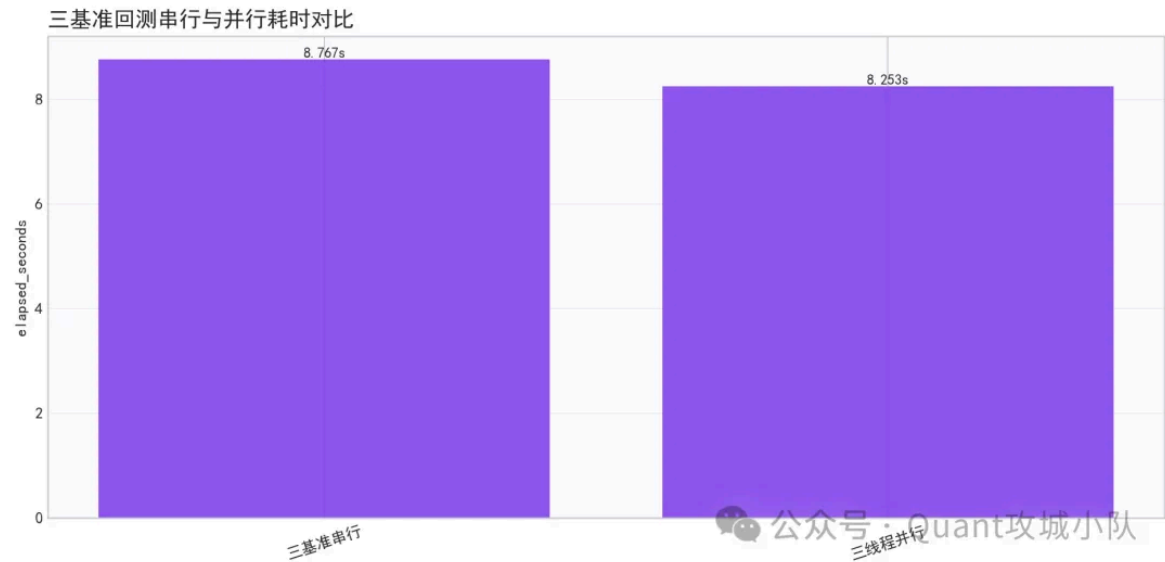
prepare_backtest_data() 只需要首次准备或增量更新；load_backtest_data() 把本地回测基础数据加载到内存。单次回测不需要反复从云端拉行情和交易限制数据，这也是日常投研能快速迭代的关键。

算法优点三：可以并行批量跑

BacktestData 的设计适合并发复用：加载一次基础数据，然后把同一份数据传给多个回测任务。常见场景包括同一个因子对多个 benchmark 跑回测、同一个因子对多个 universe 跑回测，以及不同因子的批量横向比较。

串行 vs 并行耗时

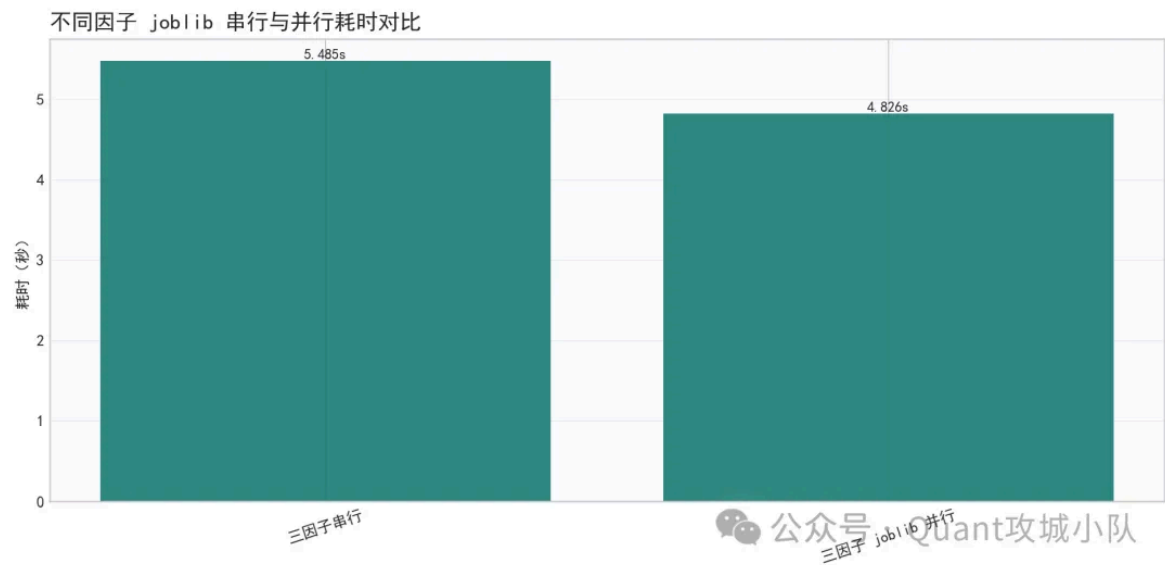
case	elapsed_seconds	benchmark	best_group
三基准串行	8.767s	000300/000905/000852	-
多线程并行	8.253s	000300/000905/000852	-



三个不同因子的批量回测，用同一套 `group_num=20`、`benchmark="000852"`、`plot="long"` 口径，分别记录串行总耗时和 `joblib` 并行总耗时。

不同因子串行 vs `joblib` 并行耗时

case	factor_count	plot_style	n_jobs	elapsed_seconds	benchmark	best_group
三因子串行	3	long	1	5.485s	000852	-
三因子 joblib 并行	3	long	3	4.826s	000852	-



并行批量跑完后，每个因子仍然保留自己的指标摘要，方便继续比较“哪个因子值得深入看”。

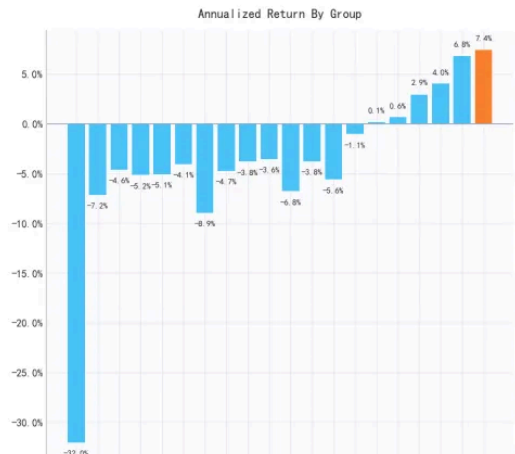
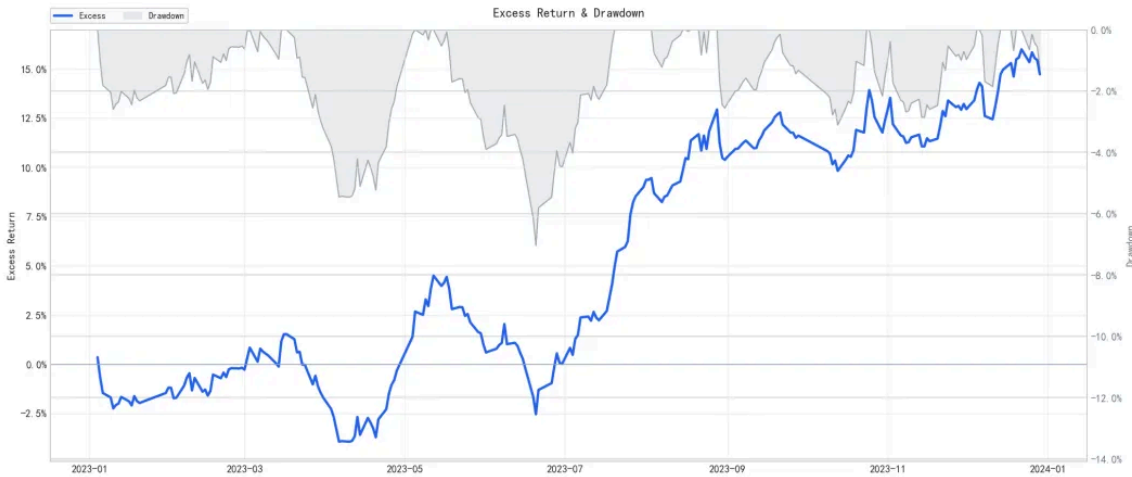
场景	因子	时间区间	交易 日数	股票 数	出图 模式	耗时	基准	最佳 组	年化超额 收益	超额 Sharpe	最大回 撤	Calmar	胜率
dtws_20	dtws_20	2023-01-01~2023-12-31	242	5121	long	4.771s	000852	19	15.40%	1.71	-7.04%	2.19	56.43%
trend_dominance_factor_20	trend_dominance_factor_20	2023-01-01~2023-12-31	242	5118	long	2.902s	000852	18	19.49%	2.62	-5.11%	3.82	57.68%
intraday_jump_intensity_factor_20	intraday_jump_intensity_factor_20	2023-01-01~2023-12-31	242	5121	long	3.801s	000852	19	12.19%	1.99	-5.73%	2.13	54.77%

```
from joblib import Parallel, delayed
```

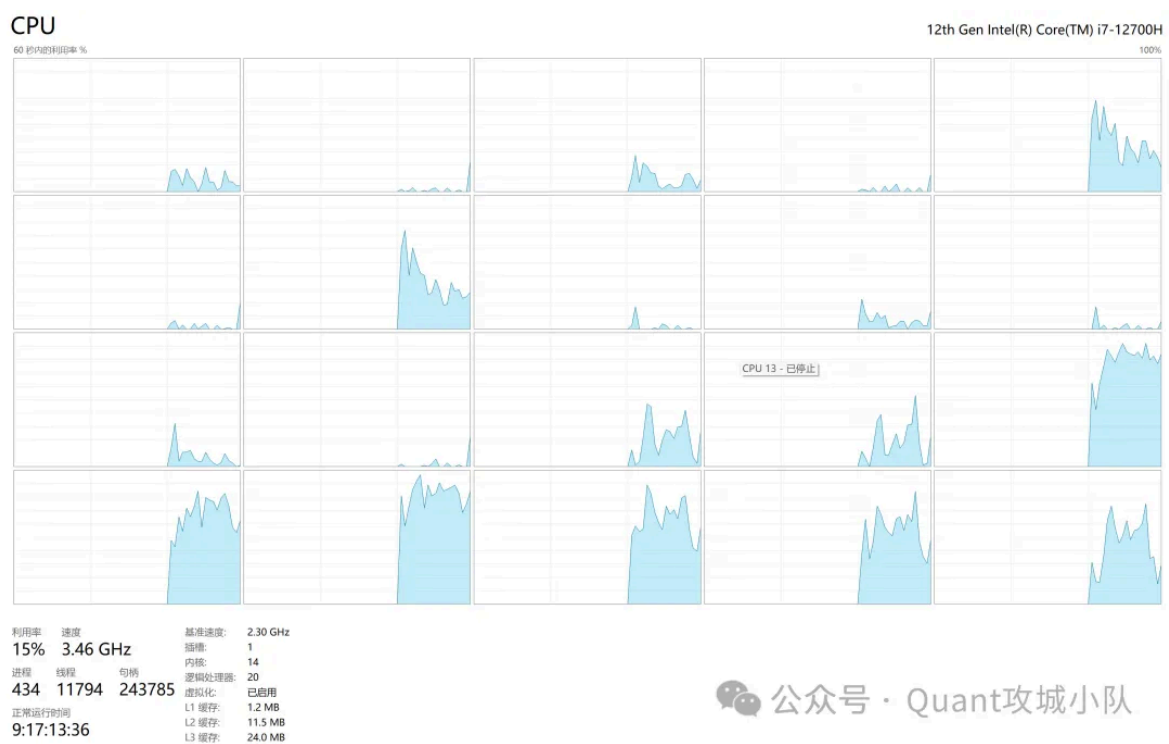
```
factor_names = [  
    "dtws_20",  
    "trend_dominance_factor_20",  
    "intraday_jump_intensity_factor_20",  
]  
  
def run_one_factor(factor_name, factor):  
    return qz.backtest(  
        factor,  
        backtest_data,  
        START_DATE,  
        END_DATE,  
        f"./output/factor_batch/{factor_name}",  
        f"{factor_name}_benchmark_000852",  
        group_num=20,  
        benchmark="000852",  
        plot="long",  
    )  
  
results = Parallel(n_jobs=3, prefer="threads")(  
    delayed(run_one_factor)(factor_name, factor_map[factor_name])  
    for factor_name in factor_names  
)
```

dtws_20_benchmark_000852 | Benchmark 000852 | VWAP Group Backtest

619	15.40%	1.71	-7.04%	2.19	56.43%	-3.32%	-5.31
Best Group	Excess Ann.	Sharpe	Max DD	Calmar	Win Rate	IC Mean	ICIR







如果读者的电脑CPU主频更高的话，计算速度可翻倍。即便是小编这个老的电脑，如果同时开20个进程计算20个因子的分组回测，平均每个因子的回测时间就妥妥压缩到1秒以内了。

常见问题

为什么回测结果为空？ 优先检查 `factor.index` 是否和交易日对齐，`factor.columns` 是否是 6 位股票代码，回测区间是否落在因子数据和基础数据交集里，以及 `universe` 是否过窄。

为什么最佳组不是最后一组？ 可能是因子方向写反、样本期内反向有效、交易成本或成交限制影响头部组合，或者因子只在某些股票池内有效。

为什么要先运行 `prepare_backtest_data()` ？ 因为回测需要行情、交易限制、复权、成分股、基准收益等本地基础数据。首次运行会下载并解压，后续运行会增量更新。

| 结尾：把时间花在因子本身

标准化的分组回测，不是为了替代研究判断，而是为了让研究判断建立在更稳定、更完整、更可复现的基础上。

当交易成本、成交约束、股票池、基准、分组数量和输出指标都由同一个函数统一处理后，研究者就可以把更多时间留给真正重要的问题：因子为什么有效？在哪些股票池里有效？对成本和参数是否敏感？能否和已有因子互补？是否有继续投入实盘验证的价值？

QuantZone SDK 想做的，就是把这些基础设施先搭好，让大家更快进入投研本身。

| 参考

- QuantZone 官方文档：<https://quantzone.tech/docs>
- PyPI quantzone：<https://pypi.org/project/quantzone/>